

Query mongodb cluster

```
curl -O
https://raw.githubusercontent.com/mongodb/docs-assets/primer-dataset/primer-dataset.json
cat primer-dataset.json | kubectl exec -it mongo-0 -- mongoimport --db test
--collection restaurants --drop
kubectl exec -it mongo-0 -- mongo test --eval "db.restaurants.find()"
```

Create work queue

```
QUEUE_POD=$(kubectl get pods -l app=work-queue,component=queue -o jsonpath='{.items[0].metadata.name}')
kubectl port-forward $QUEUE_POD 8080:8080
```

Create kuard Deployment

```
IP=$(kubectl get svc parse-server -o go-template='{
```

```
# Warn: launch after having run 'kubectl port-forward
```

```
curl -X POST \
```

```
-H "X-Parse-Application-Id: my-app-id" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"score":123,"playerName":"Sean Plott","cheat
```

```
http://${IP}:1337/parse/classes/GameScore
```

```
kubectl run requests-pod-N --image=busybox --restart Never  
--requests='cpu=1.5,memory=20Mi' -- dd if=/dev/zero of=/dev/null
```

```
# Delete all namespaces with label 'RBAC=role' to make current script idempotent

# Create namespaces 'foo' and 'bar' and add label "RBAC=role"

# Create a deployment and its related NodePort service in ns 'foo'
# for example use image dontrebootme/microbot:v1

# Create pod using image 'luksa/kubectl-proxy', and named 'shell' in ns 'bar'

# Wait for pod bar:shell to be in running state
while true
do
    sleep 2
    STATUS=$(kubectl get pods -n bar shell -o jsonpath="{.status.phase}")
    if [ "$STATUS" = "Running" ]; then
        break
    fi
done
# Access svc 'foo:microbot-service' from pod 'bar:shell'
```